

Q1) What is a parallel computer?

Solution:

A parallel computer is a collection of processing elements that cooperate to solve problems quickly.

Q2) How can you define Parallelism? Describe briefly different types of parallelism with example?

Solution:

Two/More processor (Core) executes Two/More process/events at the same time(simultaneously) is called Parallelism.

- An event typically means one of the following:

- An arithmetical operation
- A logical operation
- Accessing memory
- Performing input or output (I/O)

Types of Parallelism

- Parallelism can be examined at several levels.

- **Job level:** several independent jobs simultaneously run on the same computer system.
- **Program level:** several tasks are performed simultaneously to solve a single common problem.
- **Instruction level:** the processing of an instruction, such as adding two numbers, can be divided into sub-instructions. If several similar instructions are to be performed their sub-instructions may be overlapped using a technique called *pipelining*.
- **Bit level:** when the bits in a word are handled one after the other this is called a *bit-serial* operation. If the bits are acted on in parallel the operation is *bit-parallel*.

In this parallel processing course we shall be mostly concerned with parallelism at the program level. *Concurrent processing* is the same as parallel processing.

Q3) Give a classification of parallel machine. Define different types of parallel machine (block diagram, example).

Solution:

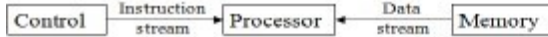
Depending on whether there is one or several of these streams we have 4 classes of computers.

- Single Instruction Stream, Single Data Stream: SISD
- Multiple Instruction Stream, Single Data Stream: MISD
- Single Instruction Stream, Multiple Data Stream: SIMD
- Multiple Instruction Stream, Multiple Data Stream: MIMD

SISD Computers

This is the standard sequential computer.

A single processing unit receives a single stream of instructions that operate on a single stream of data



Example:

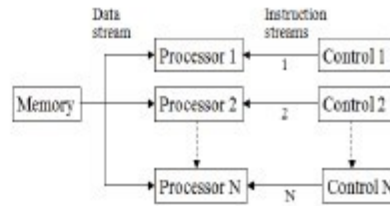
To compute the sum of N numbers $a_1, a_2, \dots, a_N$, the processor needs to gain access to memory N consecutive times. Also $N-1$ additions are executed in sequence. Therefore the computation takes $O(N)$ operations

Algorithms for SISD computers do not contain any process parallelism since there is only one processor.

26

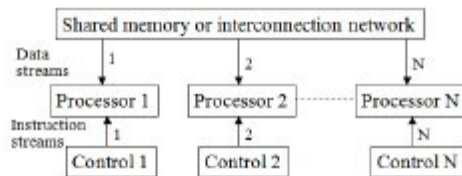
MISD Computers

N processors, each with its own control unit, share a common memory.



MIMD Computers

This is the most general and most powerful of our classification. We have N processors, N streams of instructions, and N streams of data.



Q4) What is super-scalar Execution?

Solution:

Exploits ILP within an instruction stream. Process different instructions from the same instruction stream in parallel (within a core).

Q5) What is overhead of parallel programming?

Solution:

Parallel Overhead

The amount of time required to coordinate parallel tasks, as opposed to doing useful work. Parallel overhead can include factors such as:

- Task start-up time
- Synchronizations
- Data communications
- Software overhead imposed by parallel languages, libraries, operating system, etc.
- Task termination time

Q6) What is bandwidth of memory?

Solution:

-The rate at which the memory system can provide data to a processor

-Example: 20 GB/s

Q7) What is latency of memory?

Solution:

- The amount of time for a memory request (e.g., load, store) from a processor to be serviced by the memory system
- Example: 100 cycles, 100 nsec

Q8) What is arithmetic intensity?

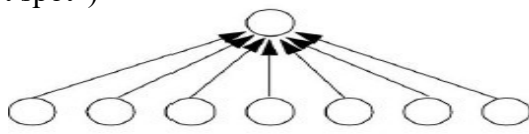
Solution:

- Ratio of math operations to data access operations in an instruction stream
- Main point: programs must have high arithmetic intensity to utilize modern processors efficiently

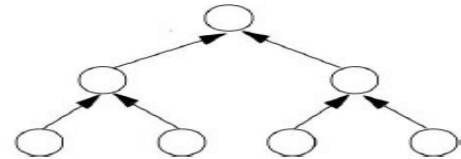
Q9) What is contention? How it can be occurred?

Solution:

- Contention occurs when many requests to a resource are made within a small window of time (the resource is a “hot spot”)



Flat communication:
potential for high contention
(but low latency if no contention)



Tree structured communication:
reduces contention
(but higher latency under no contention)

Q10) What is a parallel programming model?

Solution:

Programming models provide a way to think about the organization of parallel programs. They provide abstractions that admit many possible implementations.

Q11) Discuss the difference between abstraction and implementation in the paradigm of parallel programming. Give a general architecture for parallel system layers: interface, implementation, , ...

Solution:

A parallel programming model is an abstraction of the computer system architecture. Therefore, it is not tied to any specific machine type. However, there are many possible models for parallel computing because of the different ways several processors can be put together to build a parallel system. In addition, separating the model from its actual implementation is often difficult.

Example: ISPC: abstraction vs. implementation

▪Single program, multiple data (SPMD) programming model

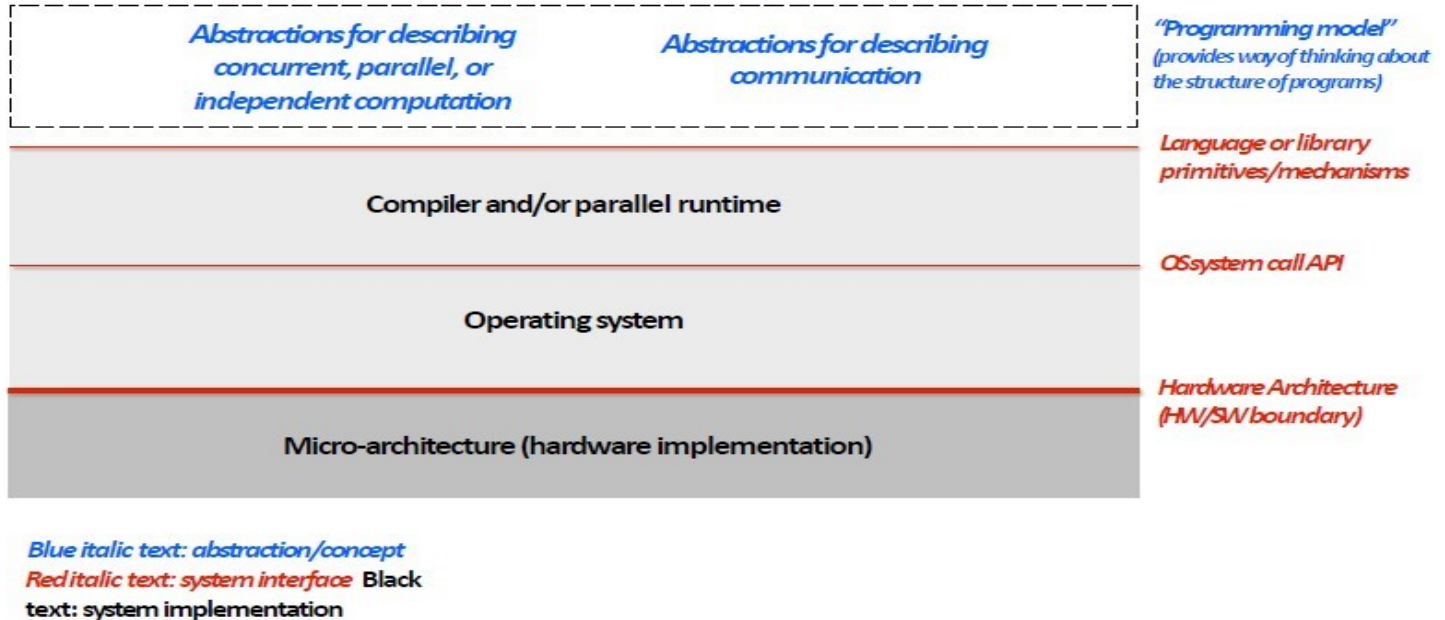
- Programmer “thinks”: running a gang is spawning programCount logical instruction streams (each with a different value of programIndex)
- This is the programming abstraction
- Program is written in terms of this abstraction

▪Single instruction, multiple data (SIMD) implementation

- ISPC compiler emits vector instructions (SSE4 or AVX) that carry out the logic performed by a ISPC gang
- ISPC compiler handles mapping of conditional control flow to vector instructions (by masking vector lanes, etc.)

System layers: interface, implementation, interface, ...

Parallel Applications



Q12) What is Instruction stream coherence, and how discuss its importance for parallel computing

Solution:

Same instruction sequence applies to all elements operated upon simultaneously Coherent execution is necessary for efficient use of SIMD processing resources

Coherent execution IS NOT necessary for efficient parallelization across cores, since each core has the capability to fetch/decode a different instruction stream

Q13) Differentiate between implicit explicit SIMD.

Solution:

"explicit SIMD": SIMD parallelization is performed at compile time

-Can inspect program binary and see instructions (vstoreps, vmulps, etc.)

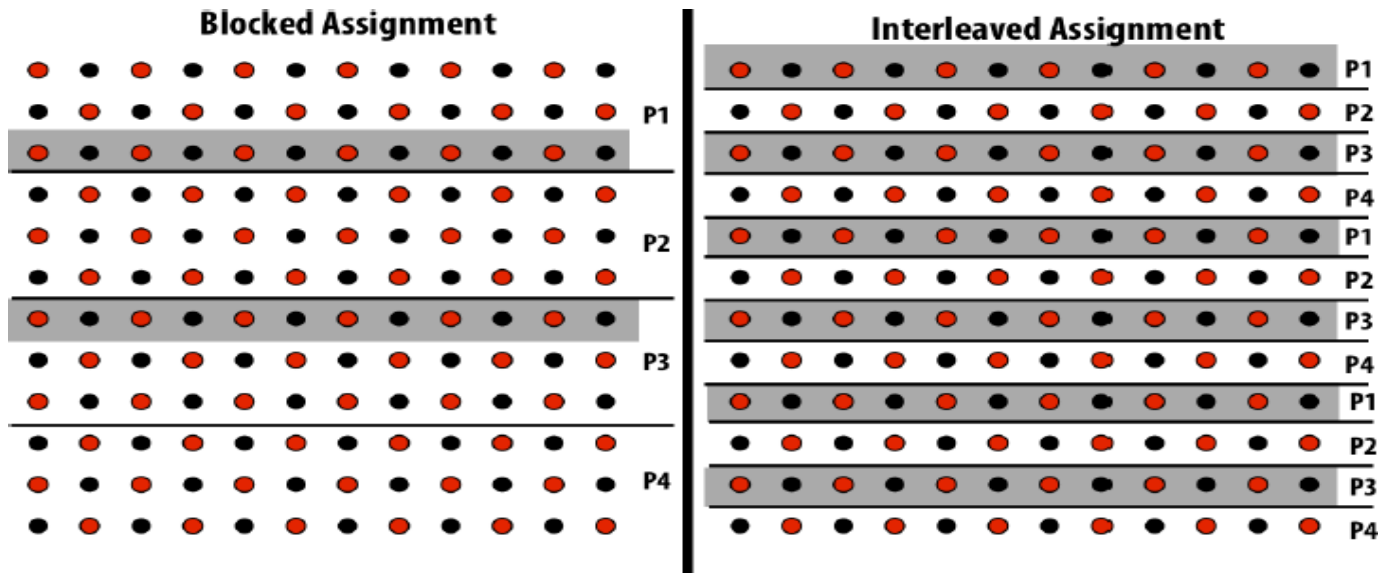
"Implicit SIMD"

Compiler generates a scalar binary (scalar instructions) But N instances of the program are *always run* together on the processor

execute(my_function, N) // execute my_function N times In other words, the interface to the hardware itself is data-parallel Hardware (not compiler) is responsible for simultaneously executing the same instruction from multiple instances on different data on SIMD ALUs

Q14) Differentiate between interleaved and blocked assignment.

Solution:



Q15) Differentiate between static and dynamic assignment.

Solution:

Static assignment (Programmer managed assignment)

Assignment of work to threads is pre determined

-Not necessarily determined at compile time (assignment algorithm may depend on runtime parameters such as input data size, number of threads, etc.)

- When work is predictable , but not all jobs have same cost (see example below)
- When statistics about execution time are known (e.g., same cost on average)

Dynamic assignment (compiler or operating system managed assignment)

Program determines assignment dynamically at runtime to ensure a well distributed load. (The execution time of tasks, or the total number of tasks, is unpredictable)

Q16) Discuss the tradeoff of choosing task size on the overall parallel performance.

Solution:

- Useful to have many more tasks* than processors (many small tasks enables good workload balance via dynamic assignment) - Motivates small granularity tasks
- But want as few tasks as possible to minimize overhead of managing the assignment -Motivates large granularity tasks
- Ideal granularity depends on many factors (must know your workload, and your machine)

Q17) What are the challenges of achieving good workload balance?

Solution:

- Want all processors working all the time (otherwise, resources are idle!)
- But want low-cost solution for achieving this balance
- Minimize computational overhead (e.g., scheduling/assignment logic)
- Minimize synchronization costs

Q18) Differentiate between synchronous and asynchronous communications.

Solution:

- Synchronous communications require some type of "handshaking" between tasks that are sharing data. This can be explicitly structured in code by the programmer, or it may happen at a lower level unknown to the programmer.
- Synchronous communications are often referred to as blocking communications since other work must wait until the communications have completed.
- Asynchronous communications allow tasks to transfer data independently from one another. For example, task 1 can prepare and send a message to task 2, and then immediately begin doing other work. When task 2 actually receives the data doesn't matter.
- Asynchronous communications are often referred to as non-blocking communications since other work can be done while the communications are taking place.
- Interleaving computation with communication is the single greatest benefit for using asynchronous communications.

Q19) Differentiate between inherent and artifactual communication.

Solution:

Inherent communication: information that fundamentally must be moved between processors to carry out the algorithm given the specified assignment (assumes unlimited capacity caches, minimum granularity transfers, etc.)

Artifactual communication: all other communication (artifactual communication results from practical details of system implementation)

Q20) Discuss the various techniques for reducing communication costs.

Solution:

Reduce overhead of communication to sender/receiver

- Send fewer messages, make messages larger (amortize overhead)
- Coalesce many small messages into large ones

▪Reduce delay

- Application writer: restructure code to exploit locality
- HW implementor: improve communication architecture

▪Reduce contention

- Replicate contended resources (e.g., local copies, fine-grained locks)
- Stagger access to contended resources

▪Increase communication/computation overlap

- Application writer: use asynchronous communication (e.g., async messages)
- HW implementor: pipelining, multi-threading, pre-fetching, out-of-order exec
- Requires additional concurrency in application (more concurrency than number of execution units)

▪Improving temporal locality

▪Improve arithmetic intensity

▪Exploiting spatial locality

Q21) Differentiate between SIMD and MIMD.

Solution:

In architectures referred to as single instruction stream, multiple data stream (SIMD), a single control unit dispatches instructions to each processing unit. In an SIMD parallel computer, the same instruction is executed synchronously by all processing units. Computers in which each processing element is capable of executing a different program independent of the other processing elements are called multiple instruction stream, multiple data stream (MIMD) computers. SIMD computers require less hardware than MIMD computers because they have only one global control unit. SIMD computers require less memory because only one copy of the program needs to be stored. In contrast, MIMD computers store the program and operating system at each processor.

Q22) Differentiate between UMA and NUMA.

Solution:

If the time taken by a processor to access any memory word in the system (global or local) is identical, the platform is classified as a uniform memory access (UMA) multicomputer. On the other hand, if the time taken to access certain memory words is longer than others, the platform is called a non-uniform memory access (NUMA) multicomputer.

Q23) Define decomposition of tasks.

Solution:

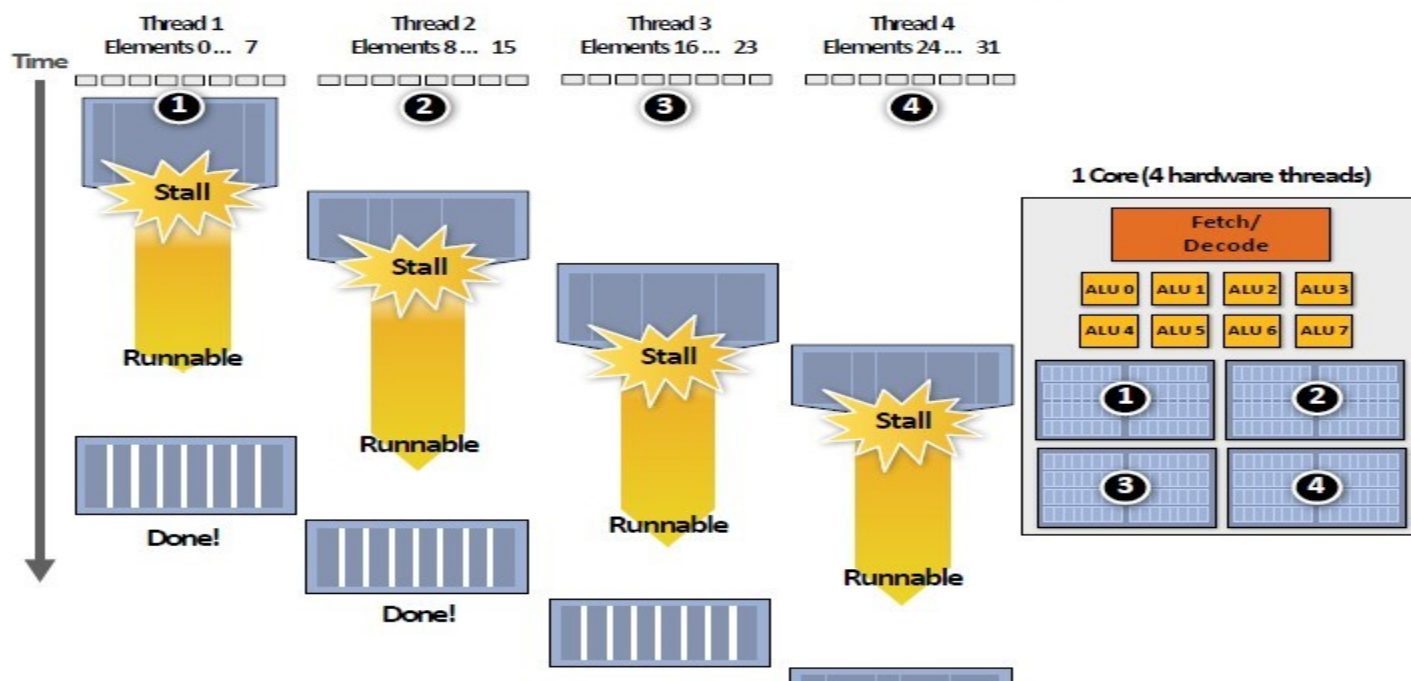
The process of dividing a computation into smaller parts, some or all of which may potentially be executed in parallel, is called decomposition. Tasks are programmer-defined units of computation into which the main computation is subdivided by means of decomposition.

Q24) Explain how multi-threading can hide stalls. Support your explanation with an example.

Solution:

- Idea: interleave processing of multiple threads on the same core to hide stalls
- Like prefetching, multi-threading is a latency hiding, not a latency reducing technique

Hiding stalls with multi-threading



Q25) Explain how memory bandwidth can limit the parallel execution. Support your explanation with an example.

Solution:

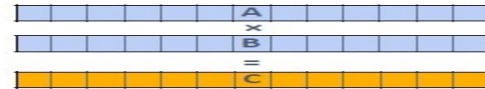
If processors request data at too high a rate, the memory system cannot keep up. No amount of latency hiding helps this. Overcoming bandwidth limits are a common challenge for application developers on throughput-optimized systems.

Thought experiment

Task: element-wise multiplication of two vectors A and B

Assume vectors contain millions of elements

- Load input A[i]
- Load input B[i]
- Compute $A[i] \times B[i]$
- Store result into C[i]



Three memory operations (12 bytes) for every MUL/MEMO

GT480 GPU can do 480 MULs per clock (@1.2 GHz)

Need ~6.4 TB/sec of bandwidth to keep functional units busy (only have 177 GB/sec)

~ 3% efficiency... but 7x faster than quad-core CPU!

(2.6 GHz Core i7 Gen 4 quad-core CPU connected to 25 GB/sec memory bus will exhibit similar efficiency on this computation)

Q26) Discuss the methods of handling the memory bandwidth limitation?

Solution:

- Organize computation to fetch data from memory less often
- Reuse data previously loaded by the same thread (traditional intra-thread temporal locality optimizations)
- Share data across threads (inter-thread cooperation)
- Request data less often (instead, do more arithmetic: it's "free")
- Useful term: "arithmetic intensity" — ratio of math operations to data access operations in an instruction stream
- Main point: programs must have high arithmetic intensity to utilize modern processors efficiently

Q27) Discuss the mechanisms for preserving atomicity in parallel programming.

Solution:

Lock/unlock mutex around a critical section

LOCK(mylock);

// criticalsection UNLOCK(mylock);

- Some languages have first-class support for atomicity of code blocks

atomic{

// criticalsection

}

- Intrinsics for hardware-supported atomic read-modify-write operations

atomicAdd(x,10);

Q28) Define granularity, fine-grained, and coarse-grained of a decomposition.

Solution:

The number and size of tasks into which a problem is decomposed determines the granularity of the decomposition. Decomposition into a large number of small tasks is called fine-grained and decomposition into a small number of large tasks is called coarse-grained.

Q29) What are the performance metrics of parallel systems?

Solution:

Execution Time

The serial runtime of a program is the time elapsed between the beginning and the end of its execution on a sequential computer. The parallel runtime is the time that elapses from the moment a parallel computation starts to the moment the last processing element finishes execution. We denote the serial runtime by TS and the parallel runtime by TP.

Total Parallel Overhead

We define overhead function or total overhead of a parallel system as the total time collectively spent by all the processing elements over and above that required by the fastest known sequential algorithm for solving the same problem on a single processing element. We denote the overhead function of a parallel system by the symbol To.

Speedup

It is defined as the ratio of the time taken to solve a problem on a single processing element to the time required to solve the same problem on a parallel computer with p identical processing elements. We denote speedup by the symbol S.

Efficiency

Efficiency is a measure of the fraction of time for which a processing element is usefully employed; it is defined as the ratio of speedup to the number of processing elements. In an ideal parallel system, speedup is equal to p and efficiency is equal to one.

Q30) State Amdahl law with derivation and example.

Solution:

- Amdahl's Law states that the maximum speedup of an algorithm is limited by the relative number of operations that must be performed sequentially, i.e., by its *serial fraction*.
- If a is the serial fraction, n is the number of operations in the sequential algorithm, and N the number of processors, then the time for the parallel algorithm is:

$$T_{\text{par}}(N) = (an + (1-a)n/N)t + C(n,N)$$

where C(n,N) is the time for overhead due to communication, load balancing, etc., and t is the time for one operation.

Derivation:

- The speed-up satisfies:

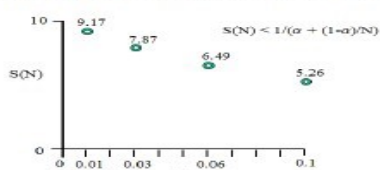
$$\begin{aligned} S(N) &= T_{\text{seq}}/T_{\text{par}}(N) = nt/[(an + (1-a)n/N)t + C(n,N)] \\ &= 1/[(a + (1-a)/N) + C(n,N)/(nt)] \\ &< 1/(a + (1-a)/N) \end{aligned}$$

- Note that as $N \rightarrow \infty$, then $S(N) \rightarrow 1/a$, so the speed-up is always limited to a maximum of $1/a$ no matter how many processors are used.

Example:

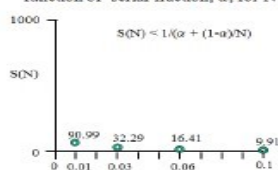
Examples of Amdahl's Law

Consider the effect of Amdahl's Law on speed-up as a function of serial fraction, α , for $N=10$ processors.



Examples of Amdahl's Law 2

Consider the effect of Amdahl's Law on speed-up as a function of serial fraction, α , for $N=1000$ processors.



If 1% of a parallel program involves serial code, the maximum speed-up is 9 on a 10-processor machine, but only 91 on a 1000-processor machine.

Q31) Define the factors that limit the speed-up of parallel algorithms.

Solution:

Optimizing the performance of parallel programs is an iterative process of refining choices for decomposition, assignment, and orchestration...

- Key goals (that are at odds with each other)

- Balance workload onto available execution resources

- Reduce communication (to avoid stalls)

- Reduce extra work (overhead) performed to increase parallelism, manage assignment, reduce communication, etc.

-- Factors That Limit Speed-up

1. Software Overhead

Even when the sequential and parallel algorithms perform the same computations, software overhead may be present in the parallel algorithm. This includes additional index calculations necessitated by how the data were decomposed and assigned to processors, and other sorts of “bookkeeping” required by the parallel algorithm but not the sequential algorithm.

02. Load Balance

Each processor should be assigned the same amount of work to do between synchronisation points. Otherwise some processors may be idle while waiting for others to catch up. This is known as load imbalance. The speedup is limited by the slowest processor.

03. Communication Overhead

Assuming that communication and calculation cannot be overlapped, then any time spent communicating data between processors reduces the speed-up.

Q32) Name and define the scalability of parallel algorithms.

Solution:

Scalability is a measure of how effectively an algorithm makes use of additional processors.

- An algorithm is said to be scalable if it is possible to keep the efficiency constant by increasing the problem size as the number of processors increases.

- An algorithm is said to be perfectly scalable if the efficiency remains constant when the problem size and the number of processors increase by the same factor.

- An algorithm is said to be highly scalable if the efficiency depends only weakly on the number of processors when the problem size and the number of processors increase by the same factor.

- Two types of scaling based on time to solution: strong scaling and weak scaling.

- Strong scaling:

- o The total problem size stays fixed as more processors are added.

- o Goal is to run the same problem size faster

- o Perfect scaling means problem is solved in $1/P$ time (compared to serial)

- Weak scaling:

- o The problem size per processor stays fixed as more processors are added. The total problem size is proportional to the number of processors used.

- o Goal is to run larger problem in same amount of time

- o Perfect scaling means problem P_x runs in same time as single processor run

Q33) Give the classification of shared memory computer with definition. What are the limitations of shared memory?

Solution:

1. Exclusive Read, Exclusive Write (EREW)

- Access to memory locations is exclusive, i.e., no 2 processors are allowed to simultaneously read from or write into the same location.

2. Concurrent Read, Exclusive Write (CREW)

- Multiple processors are allowed to read from the same location, but write is still exclusive, i.e., no 2 processors are allowed to write into the same location simultaneously.

3. Exclusive Read, Concurrent Write (ERCW)

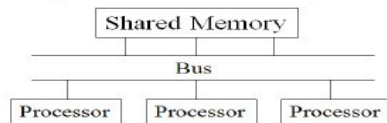
- Multiple processors are allowed to write into the same location, but read access remains exclusive.

4. Concurrent Read, Concurrent Write (CRCW)

- Both multiple read and write privileges are allowed.

Limitation of shared memory

- Shared memory computers are often implemented by incorporating a fast bus to connect processors to memory.



- However, because the bus has a finite bandwidth, i.e., it can carry only a certain maximum amount of data at any one time, then as the number of processors increase the contention for the bus becomes a problem. So it is feasible to build shared memory machines with up to only about 100 processors.

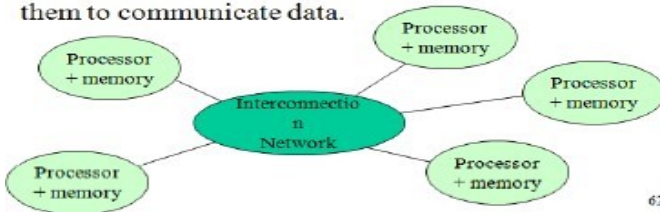
56

Q34) Define inter process communication with message passing. Differentiate between shared and distributed memory models.

Solution:

Interconnection Networks and Message Passing

In this case each processor has its own private (local) memory and there is no global, shared memory. The processors need to be connected in some way to allow them to communicate data.



62

Message Passing

- If a processor requires data contained on a different processor then it must be explicitly passed by using communication instructions, e.g., send and receive.

P1 P2
receive (x, P2) send (x, P1)

- The value x is explicitly passed from P2 to P1. This is known as *message passing*.

63

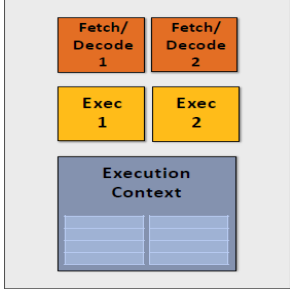
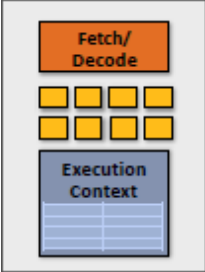
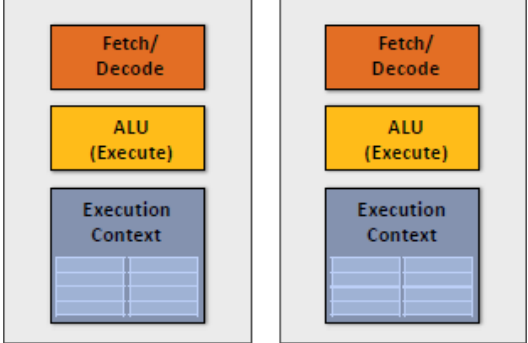
Comparison of Shared and Distributed Memory

Distributed memory	Shared memory
Large number of processors (100's to 1000's)	Moderate number of processor (10's to 100)
High peak performance	Modest peak performance
Unlimited expansion	Limited expansion
Difficult to fully utilise	Relatively easy to fully utilise
Revolutionary parallel computing	Evolutionary parallel computing

64

Q35) Explain the Super-Scalar, multi-core, and SIMD execution. Support your explanation with the help of diagram.

Solution:

Super-Scalar	Exploits ILP within an instruction stream. Process different instructions from the same instruction stream in parallel (within a core).	
SIMD	Uses multiple ALUs controlled by the same instruction stream (within a core).	
multi-core	Provides thread-level parallelism, which simultaneously executes a completely different instruction stream on each core.	

Q36) What is speedup? Differentiate Ideal speedup v/s True speedup.

Solution:

Ideal speedup: The speedup of a parallel program is

$$\text{speedup} = \frac{\text{time required for the nonparallel program execution}}{\text{time required for parallel version execution}}$$

The maximum speedup possible is always the number of processors used. However, in practice the speedup is often less than this. In the case of parallelization of a single loop, the time required for parallel execution is Time for parallel execution = time to create processes time to execute the loop + other overhead + time to destroy the children.

The time required to generate children from a single parent depends on the implementation of the process fork function. The "other overhead" involves synchronization and sequential parts of the calculation.

It is useful to consider the parallelized loop in isolation, ignoring the other overhead in the program. In this way, one can get an idea of how effectively one has parallelized the "core" of the program. The speedup so calculated will be termed the ideal speedup. The ideal speedup is the maximum possible speedup that one can attain, assuming that there is no overhead.

The existence of overhead only causes the actual speedup, obtained in a real program, to be smaller than the ideal. The ideal speedup indicates how well one has utilized the processes in the "core" parallel part of the program. As will be seen, for large problems, the true speedup can approach the ideal but will never surpass it.

Q37) Explain pthread_t, Pthread_create, pthread_kill, pthread_exit APIs of POSIX thread library.

Solution:

pthread_t is the data type used to uniquely identify a thread.

Use pthread_create(3C) to add a new thread of control to the current process

Use pthread_kill(3C) to send a signal to a thread.

Use pthread_exit(3C) to terminate a thread.

Q38) Write a short note on: 1) Contention 2) Thread V/s Process

Solution:

Contention:

Memory contention is an important performance issue in current multicore architectures. One cause of memory contention is the increasing mismatch between memory and processor performance. But in current multicore systems, sharing off-chip memory introduces the new performance problem of contention for memory bandwidth. Performance of a program degrades when multiple cores compete for off-chip memory bandwidth. Increasing the number of active cores exerts demand for off-chip memory, and results in longer memory access time and increases the total number of cycles to execute a program. With lower memory cost, larger memory capacity allows for an increase in the size of the problems that can be executed, increasing the demand for off-chip memory bandwidth and leading to contention.

Lock contention occurs when a thread attempts to acquire a lock but the lock is already held by another thread. In most cases where a thread attempts to acquire a contended lock, the thread must wait for the lock to be released before the thread can continue execution. Thus when locks are contended, threads are blocked from executing until the lock becomes free. Locks are generally used to protect shared data which may be updated. So if there is a lot of access to such data, or if a thread accessing the shared data holds the lock for a long time, then there will probably be a lot of contention.

Process

Each process provides the resources needed to execute a program. A process has a virtual address space, executable code, open handles to system objects, a security context, a unique process identifier, environment variables, a priority class, minimum and maximum working set sizes, and at least one thread of execution. Each process is started with a single thread, often called the primary thread, but can create additional threads from any of its threads.

Thread

A thread is an entity within a process that can be scheduled for execution. All threads of a process share its virtual address space and system resources. In addition, each thread maintains exception handlers, a scheduling priority, thread local storage, a unique thread identifier, and a set of structures the system will use to save the thread context until it is scheduled. The thread context includes the thread's set of machine registers, the kernel stack, a thread environment block, and a user stack in the address space of the thread's process. Threads can also have their own security context, which can be used for impersonating clients.

Q39) Compare between: 1) Problem constrained scaling, 2) Memory constrained scaling and 3) Time constrained scaling (TC)

Solution:

Problem-constrained scaling

- Focus: use a parallel computer to solve **the same problem** faster

$$\text{Speedup} = \frac{\text{time 1 processor}}{\text{time P processors}}$$

- Recall pitfalls from earlier in lecture (small problems may not be realistic workloads for large machines, big problems may not fit on small machines)
- Examples of problem-constrained scaling:
 - Almost everything we've considered parallelizing in class so far

Time-constrained scaling

- Focus: completing **more work in a fixed amount of time**
 - Execution time kept fixed as the machine (and problem) scales

$$\text{Speedup} = \frac{\text{work done by P processors}}{\text{work done by 1 processor}}$$

- How to measure "work"?
 - Challenge: "work done" may not be linear function of values of problem inputs (e.g. matrix multiplication is $O(N^3)$ work for $O(N^2)$ sized inputs)
 - One approach: "work done" is defined by execution time of same computation on a single processor (but consider effects of thrashing if problem too big)
 - Ideally, a measure of work is:
 - Simple to understand
 - Scales linearly with sequential run time (So ideal speedup remains linear in P)

Memory-constrained scaling

- Focus: **run the largest problem possible without overflowing main memory** **
- Memory per processor is held fixed (e.g., add more machines to cluster)
- Neither work nor execution time are held constant!*

$$\begin{aligned} \text{Speedup} &= \frac{\text{work(P processors)} \times \text{time (1 processor)}}{\text{time (P processors)} \times \text{work(1 processor)}} \\ &= \frac{\text{work per unit time on P processors}}{\text{work per unit time on 1 processor}} \end{aligned}$$

- Note: scaling problem size can make runtimes very large
 - Consider $O(N^3)$ matrix multiplication on $O(N^2)$ matrices

Q40) Explain Barriers in parallel computing.

Solution:

It is a synchronization type

- Usually implies that all tasks are involved
- Each task performs its work until it reaches the barrier. It then stops, or "blocks".
- When the last task reaches the barrier, all tasks are synchronized.
- What happens from here varies. Often, a serial section of work must be done. In other cases, the tasks are automatically released to continue their work.

Q41) Write a pseud code for Summing M numbers with N processors using shared and distribute memory.

Solution:

Summing m Numbers

Example: summing m numbers

On a sequential computer we have,

```
sum = a[0];
for (i=1; i<m; i++) {
    sum = sum + a[i];
}
```

Would expect the running time be roughly proportional to m. We say that the running time is $\Theta(m)$.

Summing Using Shared Memory

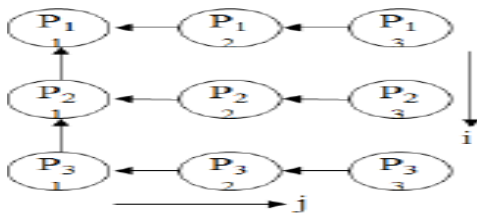
The m numbers, and the global sum, are held in global shared memory.

```
global_sum = 0;
for (each processor) {
    local_sum = 0;
    calculate local sum of m/N numbers
    LOCK
        global_sum = global_sum + local_sum;
    UNLOCK
}
```

70

Summing Using Distributed Memory

Suppose we have a square mesh of N processors.



The algorithm is as follows:

1. Each processor finds the local sum of its m/N numbers
2. Each processor passes its local sum to another processor in a coordinated way
3. The global sum is finally in processor P_{11} .

Q42) Calculate the speed-up and efficiency of a problem of summing m numbers on square mesh of N processors.

Solution:

- We now define some metrics which measure how effectively an algorithm exploits parallelism.
- **Speed-up** is the ratio of the time taken to run the best sequential algorithm on one processor of the parallel machine divided by the time to run on N processors of the parallel machine.

$$S(N) = T_{seq}/T_{par}(N)$$

- **Efficiency** is the speed-up per processor.

$$e(N) = S(N)/N = (1/N)(T_{seq}/T_{par}(N))$$

- **Overhead** is defined as $f(N) = 1/e(N) - 1$

Example:

- Suppose the best known sequential algorithm takes 8 seconds, and a parallel algorithm takes 2 seconds on 5 processors. Then

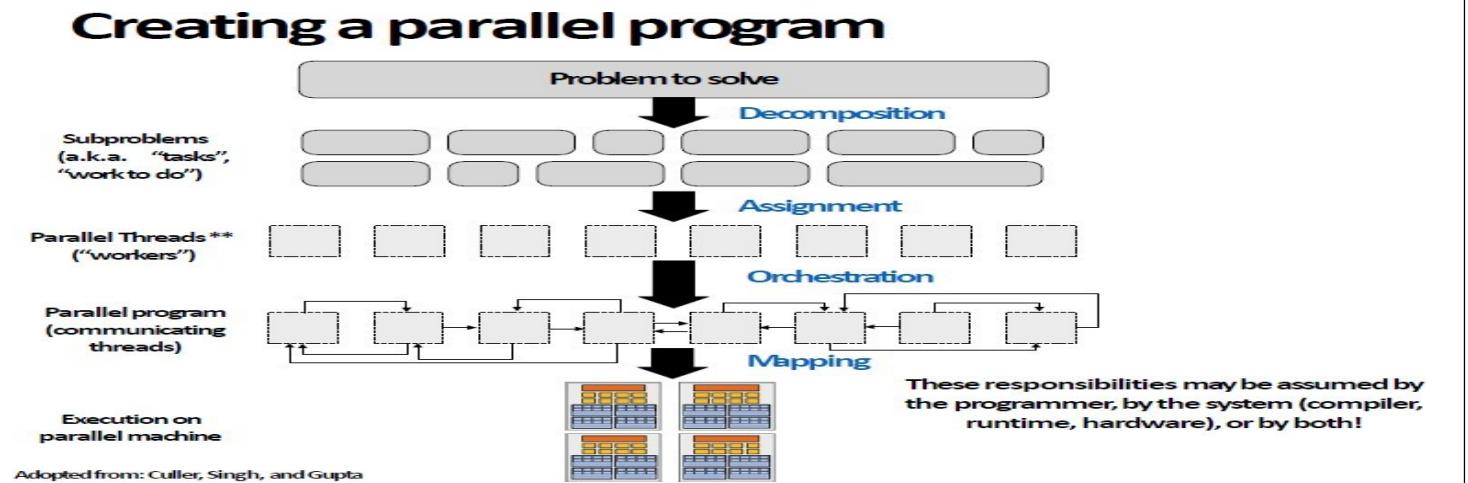
$$\text{Speed-up} = 8/2 = 4$$

$$\text{Efficiency} = 4/5 = 0.8$$

$$\text{Overhead} = 1/0.8 - 1 = 0.25$$

Q43) Describe the four major phases of creating a parallel program.

Solution:



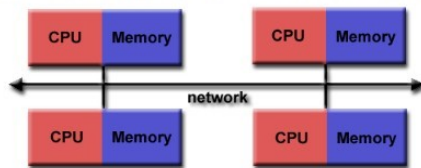
Q44) What are the major differences between message-passing and shared-address-space computers? Also outline the advantages and disadvantages of the two.

Solution:

Distributed Memory

General Characteristics:

- Like shared memory systems, distributed memory systems vary widely but share a common characteristic. Distributed memory systems require a communication network to connect inter-processor memory.



- Processors have their own local memory. Memory addresses in one processor do not map to another processor, so there is no concept of global address space across all processors.
- Because each processor has its own local memory, it operates independently. Changes it makes to its local memory have no effect on the memory of other processors. Hence, the concept of cache coherency does not apply.
- When a processor needs access to data in another processor, it is usually the task of the programmer to explicitly define how and when data is communicated. Synchronization between tasks is likewise the programmer's responsibility.
- The network "fabric" used for data transfer varies widely, though it can be as simple as Ethernet.

Advantages:

- Memory is scalable with the number of processors. Increase the number of processors and the size of memory increases proportionately.
- Each processor can rapidly access its own memory without interference and without the overhead incurred with trying to maintain global cache coherency.
- Cost effectiveness: can use commodity, off-the-shelf processors and networking.

Disadvantages:

- The programmer is responsible for many of the details associated with data communication between processors.
- It may be difficult to map existing data structures, based on global memory, to this memory organization.
- Non-uniform memory access times - data residing on a remote node takes longer to access than node local data.

Shared Memory

General Characteristics:

- Shared memory parallel computers vary widely, but generally have in common the ability for all processors to access all memory as global address space.
- Multiple processors can operate independently but share the same memory resources.
- Changes in a memory location effected by one processor are visible to all other processors.
- Historically, shared memory machines have been classified as **UMA** and **NUMA**, based upon memory access times.

Uniform Memory Access (UMA):

- Most commonly represented today by **Symmetric Multiprocessor (SMP)** machines
- Identical processors
- Equal access and access times to memory
- Sometimes called CC-UMA - Cache Coherent UMA. Cache coherent means if one processor updates a location in shared memory, all the other processors know about the update. Cache coherency is accomplished at the hardware level.

Non-Uniform Memory Access (NUMA):

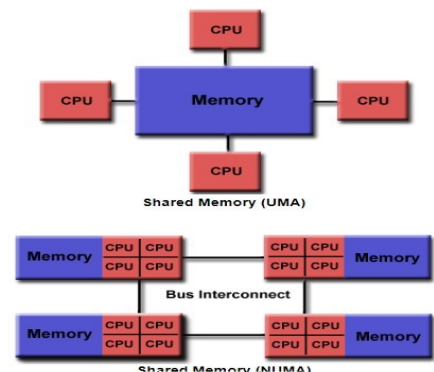
- Often made by physically linking two or more SMPs
- One SMP can directly access memory of another SMP
- Not all processors have equal access time to all memories
- Memory access across link is slower
- If cache coherency is maintained, then may also be called CC-NUMA - Cache Coherent NUMA

Advantages:

- Global address space provides a user-friendly programming perspective to memory
- Data sharing between tasks is both fast and uniform due to the proximity of memory to CPUs

Disadvantages:

- Primary disadvantage is the lack of scalability between memory and CPUs. Adding more CPUs can geometrically increase traffic on the shared memory-CPU path, and for cache coherent systems, geometrically increase traffic associated with cache/memory management.
- Programmer responsibility for synchronization constructs that ensure "correct" access of global memory.



Q45) Explain why the performance of hardware multithreaded processing core might degrade if it had large caches and it ran many threads.

Solution:

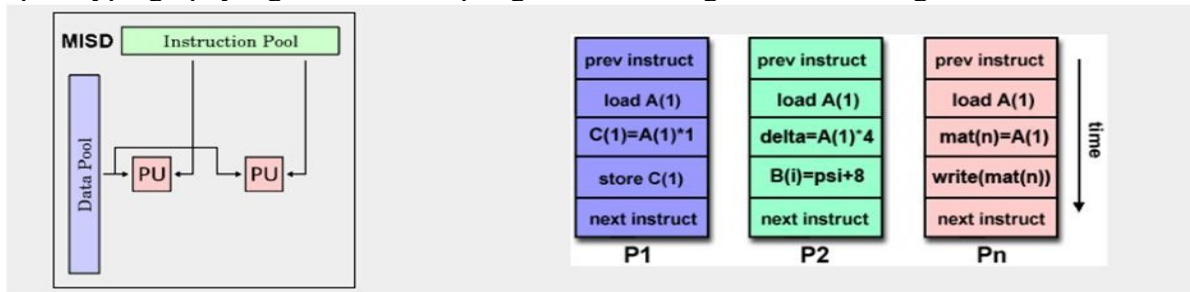
Hardware multithreading provides a means for systems to continue doing useful work when the task being currently executed has stalled—for example, if the current task has to wait for data to be loaded from memory. Instead of looking for parallelism in the currently executing thread, it may make sense to simply run another thread. Of course, in order for this to be useful, the system must support very rapid switching between threads. For example, in some older systems, threads were simply implemented as processes, and in the time it took to switch between processes, thousands of instructions could be executed.

More threads → larger working set → less cache space per thread

Q46) In our discussion of parallel hardware, we used Flynn's taxonomy to identify three types of parallel systems: SISD, SIMD, and MIMD. None of our systems were identified as multiple instruction, single data, or MISD. How would an MISD system work? Give an example.

Solution:

- Multiple Instruction: Each processing unit operates on the data independently via separate instruction streams.
- Single Data: A single data stream is fed into multiple processing units.
- Few (if any) actual examples of this class of parallel computer have ever existed.
- Some conceivable uses might be:
 - multiple frequency filters operating on a single signal stream
 - multiple cryptography algorithms attempting to crack a single coded message



Q47) Discuss the concept of writing fork-join programs, with the help of an example.

Solution:

Fork-join pattern

- Natural way to express independent work in divide-and-conquer algorithms
- This lecture's code examples will be in Cilk Plus
 - C++ language extension
 - Originally developed at MIT, acquired by Intel
 - But Intel is deprecating it. Best to stick with MIT version

cilk_spawn foo(args); "fork" (create new logical thread of control)

Semantics: invoke foo, but unlike standard function call, caller may continue executing asynchronously with execution of foo.

cilk_sync; "join"

Semantics: returns when all calls spawned by current function have completed. ("sync up" with the spawned calls)

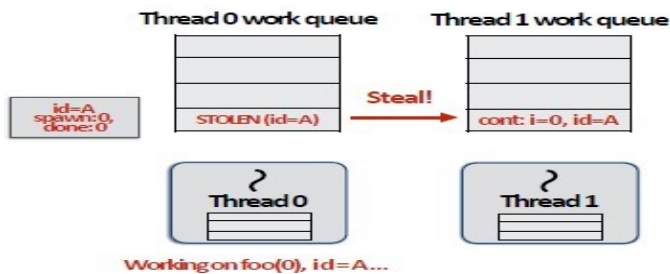
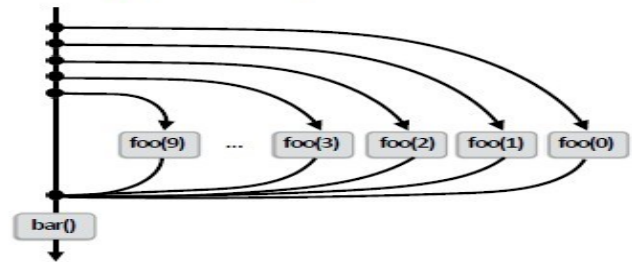
Note: there is an implicit **cilk_sync** at the end of every function that contains a **cilk_spawn** (implication: when a Cilk function returns, all work associated with that function is complete)

Q48) What is Greedy join scheduling policy?

Solution:

Implementing sync: greedy policy

```
block(id: A)
{
  for (int i=0; i<10; i++) {
    cilk_spawn foo(i);
  }
  cilk_sync; Sync for all calls spawned within block A
  bar();
}
```



Example 2: "greedy" policy

- When thread that initiates the fork goes idle, it looks to steal new work
- Last thread to reach the join point continues execution after sync

Q49) Is a program that obtains linear speedup strongly scalable? Explain your answer.

Solution:

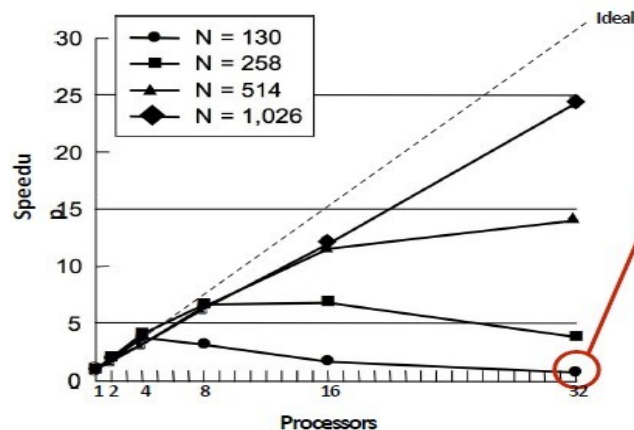
Suppose we run a parallel program with a fixed number of processes/threads and a fixed input size, and we obtain an efficiency E . Suppose we now increase the number of processes/threads that are used by the program. If we can find a corresponding rate of increase in the problem size so that the program always has efficiency E , then the program is scalable.

Strong scaling

- Runtime of X on P processors, vs. of X on 1 processor
- Goal = P
- Does having more processors get job done faster?

Pitfalls of fixed problem size speedup analysis

Ocean execution on 32 processor SGI Origin 2000



Nobenefit! (slight slowdown)

Problem size is just too small for the machine (large communication-to-computation ratio)

Scaling the performance of small problem may not be all that important anyway (it might already execute fast enough on a single core)

258 x 258 grid on 32 processors: ~ 310 grid cells per processor

1K x 1K grid on 32 processors: ~ 32K grid cells per processor

Q50) We have two computers A and B. Computer A has a clock cycle of 1 ns and performs on average 2 instructions per cycle. Computer B, instead, has a clock cycle of 600 ps and performs on average 1.25 instructions per cycle. Consider the program P, which requires the execution of the same number of instructions in both computers:

- Which computer is faster for this program?
- What if P, when executed in Computer B, performs 10% more instructions than when executed in Computer A?

Solution:

Computer A performs $\frac{2 \text{ instructions}}{1 \text{ cycle}} \times \frac{1 \text{ cycle}}{10^{-9} \text{ seconds}} = 2 \times 10^9 \frac{\text{instructions}}{\text{second}}$

Computer B performs $\frac{1.25 \text{ instructions}}{1 \text{ cycle}} \times \frac{1 \text{ cycle}}{600 \times 10^{-12} \text{ seconds}} = 2.08 \times 10^9 \frac{\text{instructions}}{\text{second}}$

Computer B performs more instructions per second, thus it is the fastest for this program.

Now, let's n be the number of instructions required by Computer A, and $1.1 \times n$ the number of instructions required by Computer B. The program will take $\frac{n}{2 \times 10^9}$ seconds in Computer A and $\frac{n}{1.89 \times 10^9}$ seconds in Computer B. Therefore, in this scenario, Computer A executes the program faster.

Q51) Assume the runtime of a program is 100 seconds for a problem of size 1. The program consists of an initialization phase which lasts for 10 seconds and cannot be parallelized, and a problem solving phase which can be perfectly parallelized and grows quadratically with increasing problem size.

- What is the speedup for the program as a function of the number of processors p and the problem size n .
- What is the execution time and speedup of the program with problem size 1, if it is parallelized and run on 4 processors?
- What is the execution time of the program if the problem size is increased by a factor of 4 and it is run on 4 processors? And on 16 processors? What is the speedup of both measurements?

Solution.

The application has an inherently sequential part (c_s) that takes 10 seconds, and a parallelizable part (c_p) that takes 90 seconds for problem size 1. Since, the parallelizable part grows quadratically with the problems size, we can model T_1 (execution time in 1 processor) as:

$$c_s + c_p \times n^2.$$

The function of the speedup is, thus,

$$S(p, n) := \frac{T(1, n)}{T(p, n)} := \frac{c_s + c_p \times n^2}{c_s + (c_p \times n^2)/p} \equiv \frac{10 + 90 \times n^2}{10 + (90 \times n^2)/p}.$$

For problem size 1 ($n = 1$) and 4 processors ($p = 4$), the execution time is 32.5 seconds. The achieved speedup is 3.08.

Finally, if problem size is increased to 4, the execution time and speedup using 4 and 16 processors is:

- 4 processors: 370 seconds, speedup of 3.92.
- 16 processors: 100 seconds, speedup of 14.5.

Q52) Assume an application where the execution of floating-point instructions on a certain processor P consumes 60% of the total runtime. Moreover, let's assume that 25% of the floating-point time is spent in square root calculations.

- Based on some initial research, the design team of the next-generation processor P2 believes that they could either improve the performance of all floating point instructions by a factor of 1.5 or alternatively speed up the square root operation by a factor of 8. From which design alternative would the aforementioned application benefit the most?
- Instead of waiting for the next processor generation the developers of the application decide to parallelize the code. What speedup can be achieved on a 16-CPU system, if 90% of the entire program can be perfectly parallelized? What fraction of the code has to be parallelized to get a speedup of 10?

Solution.

Amdahl's law: $S_p(n) := \frac{1}{\beta + (1-\beta)/p}$.

- Improvement 1 (all fp instructions sped up by a factor 1.5). Sequential part (β): 0.4. $p = 1.5$. The application would observe a total speedup of:

$$S_p(n) := \frac{1}{\beta + (1-\beta)/p} = \frac{1}{0.4 + (1-0.4)/1.5} = 1.25.$$

- Improvement 2 (square root instructions sped up by a factor of 8). Sequential part (β): 0.4 + 0.45. $p = 8$. The application would observe a total speedup of:

$$S_p(n) := \frac{1}{\beta + (1-\beta)/p} = \frac{1}{.85 + (1-0.85)/8} = 1.15.$$

Thus, the application would benefit the most from the first alternative.

Parallelization of code. The speedup achieved on a 16-CPU system is:

$$S_p(n) := \frac{1}{\beta + (1-\beta)/p} = \frac{1}{0.1 + (1-0.1)/16} = 6.4.$$

To attain a speedup of 10, a 96% of the code would need to be perfectly parallelizable. This value is obtained by solving the equation:

$$10 = \frac{1}{\beta + (1-\beta)/16}.$$

Q53) Consider a computer that has a peak performance of 8 GFlops/s. An application running on this computer executes 15 TFlops, and takes 1 hour to complete.

- How many GFlops/s did the application attain?
- Which efficiency did it achieve?

Solution.

The application attained: $\frac{15 \text{ TFlops/s}}{3600 \text{ s}} = 4.17 \text{ GFlops/s}$.

The achieved efficiency is: $\frac{4.17 \text{ GFlops/s}}{8 \text{ GFlops/s}} = 52\%$.

Q54) A given program was run on a node consisting of 4 multi-core processors, each of which comprises 10 cores. The peak performance of the node is 320 GFlops/s. To study the scalability of the program, it was run using 1, 2, 4, 8, 16, 24, 32 and 40 of the cores, for fixed problem size. For this problem size, the program executes 22×10^{12} ops. The execution time for each number of cores is collected in the below table. Complete the table with the performance, speedup, efficiency, and parallel efficiency for each case.

Solution:

# Cores	Time (s)	Perf. (GF/s)	Eff.	Speedup	Par. Eff.
1	34566.23	0.64	0.08	1.00	1.00
2	17133.64	1.28	0.08	2.02	1.01
4	8535.92	2.58	0.08	4.05	1.01
8	4326.95	5.08	0.08	7.99	1.00
16	2206.02	9.97	0.08	15.67	0.98
24	1531.88	14.36	0.07	22.56	0.94
32	1133.53	19.41	0.08	30.49	0.95
40	943.30	23.32	0.07	36.64	0.92

Q55) A program P consists of parts A, B and C. Part A is not parallelized while parts B and C are parallelized. The program is run on a computer with 10 cores. For a given problem size, the execution of the program in 1 of the cores takes 10, 120, and 53 seconds, for parts A, B, and C, respectively. Answer the following questions:

- Which is the minimum execution time we can attain for each part if we now execute the same program with same problem sizes on 5 of the cores? What is the best speedup we can expect for the entire program?
- What if we run it using the 10 cores?

Solution.

The minimum possible execution time for each part, for 5 and 10 cores, is given in Tab.

# Cores	Part A(s)	Part B (s)	Part C(s)
1	17	120	53
5	17	24	10.6
10	17	12	5.3

Table Minimum possible timings for program *P* using 1, 5 and 10 cores.

The best speedup we can achieve in each case is:

- For 5 cores: $S_5 = \frac{T_1}{T_5} = \frac{190}{17+173/5} = 3.68$
- For 10 cores: $S_{10} = \frac{T_1}{T_{10}} = \frac{190}{17+173/10} = 5.54$

The speedup achieved in practice is:

- For 5 cores: $S_5 = \frac{T_1}{T_5} = \frac{190}{68} = 2.79$
- For 10 cores: $S_{10} = \frac{T_1}{T_{10}} = \frac{190}{43} = 4.42$

Q56) Given the following program parallelized with OpenMP:

```
double calculate_pi(double step) {
    int i;
    double x, sum = 0.0;
    #pragma omp parallel for reduction(+: sum) private(x)
    for (i=0; i<1000000; ++i) {
        x = (i-0.5) * step;
        sum += 2.0 / (1.0 + x*x);
    }
    return step * sum;
}
```

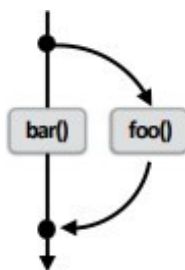
Write an equivalent version of the program without the reduction annotation.

Solution:

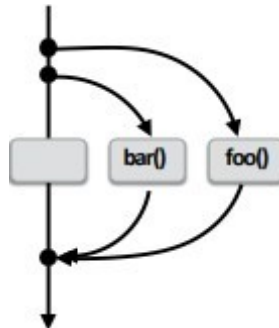
```
double calculate_pi(double step) {
    int i;
    double x, sum=0.0;
    int nth = omp_get_num_threads();
    double * sumv = malloc(sizeof(double) * nth);
    for (i=0; i<nth; ++i) sumv[i] = 0.0;

    #pragma omp parallel for private(x)
    for (i=0; i<1000000; ++i) {
        int id = omp_get_thred_num();
        x = (i-0.5) * step;
        sumv[id] += 2.0 / (1.0 + x*x);
    }
    for (i=0; i<nth; ++i) sum+=sumv[i];
    return step * sum;
}
```

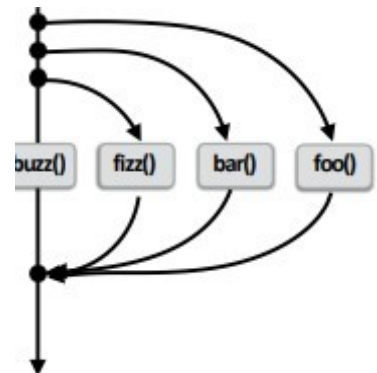
Q57) Write basic Cilk Plus examples that result to the following scenarios



(a)



(b)



(c)

Solution:

<pre>cilk_spawnfoo(); bar(); cilk_sync;</pre>	<pre>cilk_spawnfoo(); cilk_spawnbar(); cilk_sync;</pre>	<pre>cilk_spawnfoo(); cilk_spawn bar(); cilk_spawnfizz(); buzz(); cilk_sync;</pre>
---	---	--

Q58) Given the following sequential program for the particle binning example:

0	1	2	3
3 4 5	5	1 6 2	4 7
8	9 0	10	11
12	13	14	15

```
list cell_lists[16];    // 2D array of lists

for each particle p
    c = compute cell containing p
    append p to cell_lists[c]
```

Give at least three parallel versions of the program, and compare the overall performance of those versions in terms of the various metrics such as workload balance, communication, contention, ..etc.

Solution:

Parallel implementations of particle binning

Sequential algorithm

```
list cell_lists[16];    // 2D array of lists

for each particle p
    c = compute cell containing p
    append p to cell_lists[c]
```

Implementation 1: Parallel over particles

```
list cell_list[16];    // 2D array of lists
lock cell_list_lock;

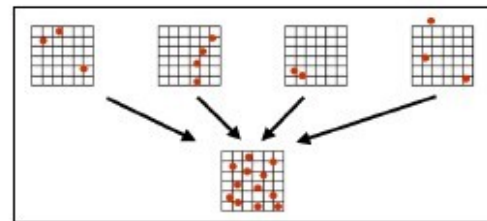
for each particle p    // in parallel
    c = compute cell containing p
    lock(cell_list_lock)
    append p to cell_list[c]
    unlock(cell_list_lock)
```

Implementation 2: Parallel over grid cells

```
list cell_lists[16];    // 2D array of lists

for each cell c        // in parallel
    for each particle p // sequentially
        if (p is within c)
            append p to cell_lists[c]
```

Implementation 3: build separate grids and merge



Implementation 4: data-parallel sort

Step 1: compute cell containing each particle

Array index: 0 1 2 3 4 5
result: 9 6 6 4 6 4

Step 2: sort results by cell

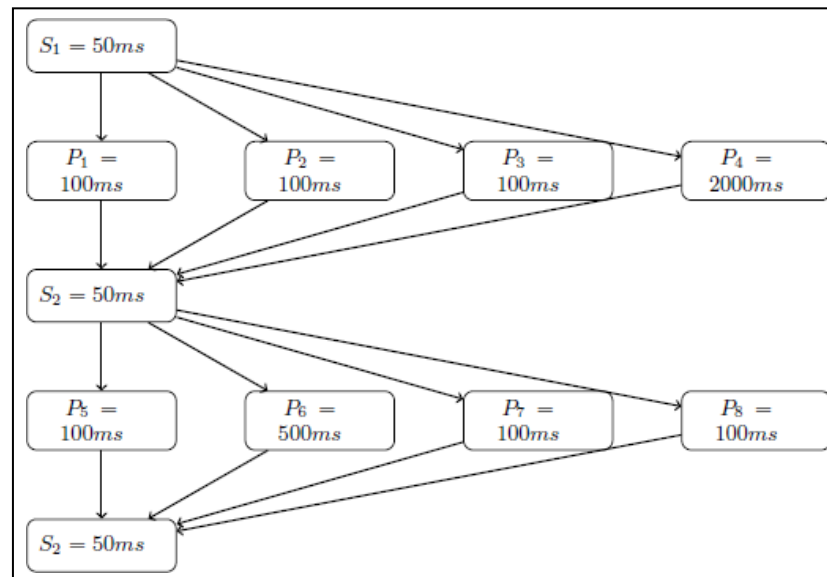
Array index: 3 5 1 2 4 0
result: 4 4 6 6 6 9

Step 3: find start/end of each cell

```
cell = result[index]
if (index == 0 || cell != result[index-1]) {
    cell_starts[cell] = index;
    if (index > 0) // special case for first cell
        cell_ends[result[index-1]] = index;
}
if (index == numParticles-1) // special case for last cell
    cell_ends[cell] = index+1;
```

Common pitfall: compare parallel program speedup to parallel algorithm running on one core (easier to make yourself look good)

Q59) Examine the task graph shown below. Each task, which you can think of as an async, is labeled with its runtime. Answer the following four questions about the program's runtime. In all cases you may ignore any work scheduling or task spawning overheads.

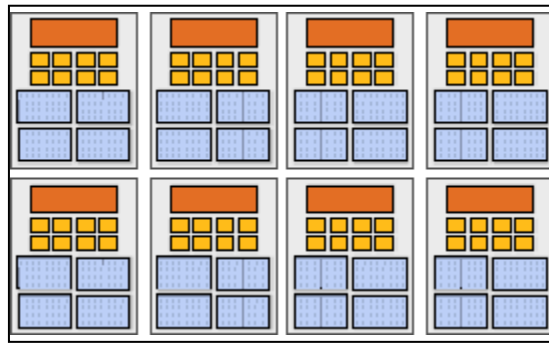


- Assuming a single worker thread (X10 NTHREADS=1) what is the runtime of this program?
- What is the speedup when X10 NTHREADS=8??
- If each parallel task ($P_N = X$ ms) were parallelized further to become two parallel tasks ($Q_N = \frac{X}{2}$ ms, $R_N = \frac{X}{2}$ ms), and again run with X10 NTHREADS=8, what would the runtime be? What is the speedup be relative to the previous run?? Why is the speedup not 2X?

Solution:

- Assuming a single worker thread (X10_NTHREADS=1) what is the runtime of this program?
The single worker thread must execute all of the tasks serially, so
 $T_1 = 50 + (2000 + 100 + 100 + 100) + 50 + (100 + 500 + 100 + 100) + 50 = 3250ms$
- What is the speedup when X10_NTHREADS=8? The stages are now executed in parallel, with the duration of the stage determined by the slowest task: $T_8 = 50 + (2000) + 50 + (500) + 50 = 2650ms$, so the speedup is $\frac{T_1}{T_8}$ or 1.226.
- If each parallel task ($P_N = Xms$) were parallelized further to become two parallel tasks ($Q_N = \frac{X}{2}ms, R_N = \frac{X}{2}ms$), and again run with X10_NTHREADS=8, what would the runtime be? What is the speedup be relative to the previous run? The slowest task in the parallel stage is now twice as fast meaning: $T'_8 = 50 + (\frac{2000}{2}) + 50 + (\frac{500}{2}) + 50 = 1400ms$, so the speedup is $\frac{T_8}{T'_8}$ or 1.89.
- Why is the speedup not 2x? This is due to Amdahl's Law. To achieve a 2x overall speedup, the entire program would need to be sped up by a factor of two, but only a portion of the program was. The serial tasks were unchanged.

Q60) For the below multi-core processor, find the followings.



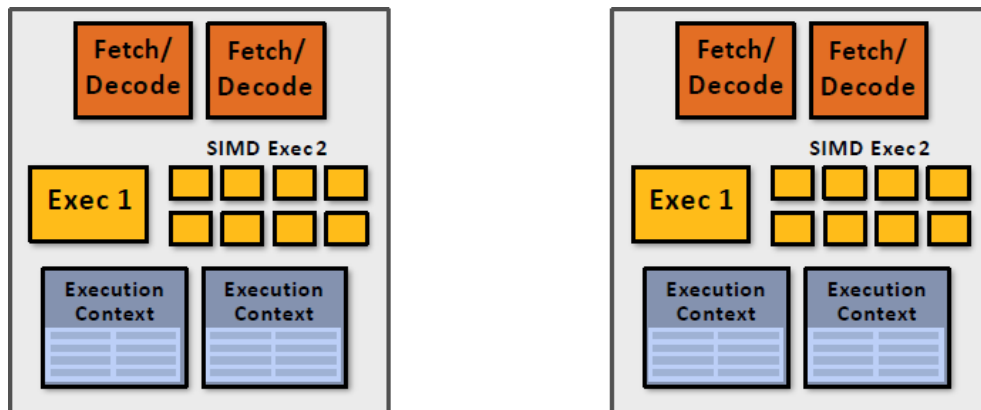
- Number of ALUs per core and then total number of ALUs.
- Number of threads per core.
- Number of simultaneous and concurrent instruction streams.
- Number of independent pieces of work is needed to run chip with maximal latency hiding ability.

Solution:

- 8, 64
- 4
- 8, 32
- 256

Q61) Design a multi-threaded, superscalar dual-core processor. The processor executes up to two instructions per clock from one instruction stream on each core (one SIMD instruction + one scalar instruction). Also, it can switch to execute the other instruction stream when faced with stall

Solution:



Q62) For achieving a speedup up to 80 with 100 processors. What original computation fraction should be parallel?

Solution:

0.9975

Q63) If 90% of the computation can be parallelized, what is the maximum speedup achievable using 8 processors?

Solution:

4.705882353

Q64) The pseudo code for sequential algorithm is provided below for a 2D-grid based solver (partial differential equation (PDE) on (N+2) x (N+2) grid).

```
const int n;
float* A; // assume allocated to grid of N+2 x N+2 elements

void solve(float* A) {
    float diff, prev;
    bool done = false;

    while (!done) { // outermost loop: iterations
        diff = 0.f;
        for (int i=1; i<=n i++) { // iterate over non-border points of grid
            for (int j=1; j<=n; j++) {
                prev = A[i,j];
                A[i,j] = 0.2f * (A[i,j] + A[i,j-1] + A[i-1,j] +
                                A[i,j+1] + A[i+1,j]);
                diff += fabs(A[i,j] - prev); // compute amount of change
            }
        }
        if (diff/(n*n) < TOLERANCE) // quit if converged
            done = true;
    }
}
```

- Write the pseudo code for the shared address space solver.
- Compare between the shared address space and message passing programming models in terms of the computation, communication, and synchronization.

Solution:

a)

Shared address space solver (pseudocode in SPMD execution model)

```
int n; // grid size
bool done = false;
float diff = 0.0;
LOCK myLock;
BARRIER myBarrier;

// allocate grid
float* A = allocate(n+2, n+2);

void solve(float* A) {
    int threadId = getThreadId();
    int myMin = 1 + (threadId * n / NUM_PROCESSORS);
    int myMax = myMin + (n / NUM_PROCESSORS);

    while (!done) {
        diff = 0.f;
        barrier(myBarrier, NUM_PROCESSORS);
        for (i=myMin to myMax) {
            for (j = red cells in this row) {
                float prev = A[i,j];
                A[i,j] = 0.2f * (A[i-1,j] + A[i,j-1] + A[i,j] +
                                A[i,j+1], A[i+1,j]);
                lock(myLock);
                diff += abs(A[i,j] - prev));
                unlock(myLock);
            }
        }
        barrier(myBarrier, NUM_PROCESSORS);
        if (diff/(n*n) < TOLERANCE) // check convergence, all threads get same answer
            done = true;
        barrier(myBarrier, NUM_PROCESSORS);
    }
}
```

Grid solver example from: Culler, Singh, and Gupta

b)

	Computation	Communication	Synchronization
Shared Address Space	Value of threadId is different for each SPMD instance: use value to compute region of grid to work on	Implicit in loads/stores to shared variables	Mutual exclusion required for shared variables (e.g., via locks) Barriers used to express dependencies (between phases of computation)
Message Passing	Array indexing is relative to local address space (not global grid coordinates)	Performed by sending and receiving messages Bulk transfer: communicate entire rows at a time (not individual elements)	Performed by sending and receiving messages Think of how to implement mutual exclusion, barriers, flags using messages